Large Language Machine — Troubleshooting Guide

Oct 15, 2026 · @Abdul-Lateef

Stuck on "This will take 2-3 minutes"

This screen means the main script reached the broker on the PC, but **no message has arrived from any of the four Pis**. The startup checks table only replaces this screen when the first sensor message arrives. The Secret Rules reader sends one about every 1.5 seconds once it is connected. If the screen is still there after 5 minutes, the break is between the Pis and the PC.

Work through these four checks in order. None of them need a password if the PC logs in to its desktop automatically.

1. **Look at the Activate button light.**
   - *Repeating 2 slow + 6 fast flashes*: the button Pi is running but cannot reach the broker. Go to check 2.
   - *Off and never flashing*: the Pis may have no power. Check the *2. Readers* socket and the small LEDs on each Pi.
   - *Solid*: the Pis can reach the broker. Go to check 4.
2. **Run network checks from the PC itself.** Plug in the internal keyboard and press **F11**. Leave the script running and open a second terminal (**Ctrl+Alt+T**, or from the dock on the right). Then run:

```
ip -4 addr                      # must include 192.168.1.10
ss -ltn | grep 2883             # must show 0.0.0.0:2883 or *:2883
ping -c 2 192.168.1.1           # the router
# the four Pis:
for i in 11 12 13 14; do ping -c 2 -W 2 192.168.1.$i; done
```

3. **Match the result to a fix:**

What the checks show	Fault	Fix
No `192.168.1.10` in `ip -4 addr`	The PC has a different address, so the Pis look in the wrong place	Reserve 192.168.1.10 for the PC in the router's DHCP settings (or set it as a static IP), then reboot everything

What the checks show	Fault	Fix
`ss` shows only `127.0.0.1:2883` or `[::1]:2883`	The broker only accepts connections from the PC itself	Add `listener 2883 0.0.0.0` and `allow_anonymous true` to a file in `/etc/mosquitto/conf.d/`, then `sudo systemctl restart mosquitto`
The router does not answer	The PC is not on the machine's network	Check the router has power, and the PC's Wi-Fi or Ethernet
The router answers but some or all Pis do not	Those Pis are not on the network	Check the router's client list, Pi power, and that the router's 2.4 GHz Wi-Fi is on
Everything answers and `ss` looks right	Firewall, or the reader programs are not running	`sudo ufw status`, then Step 3c on a Pi

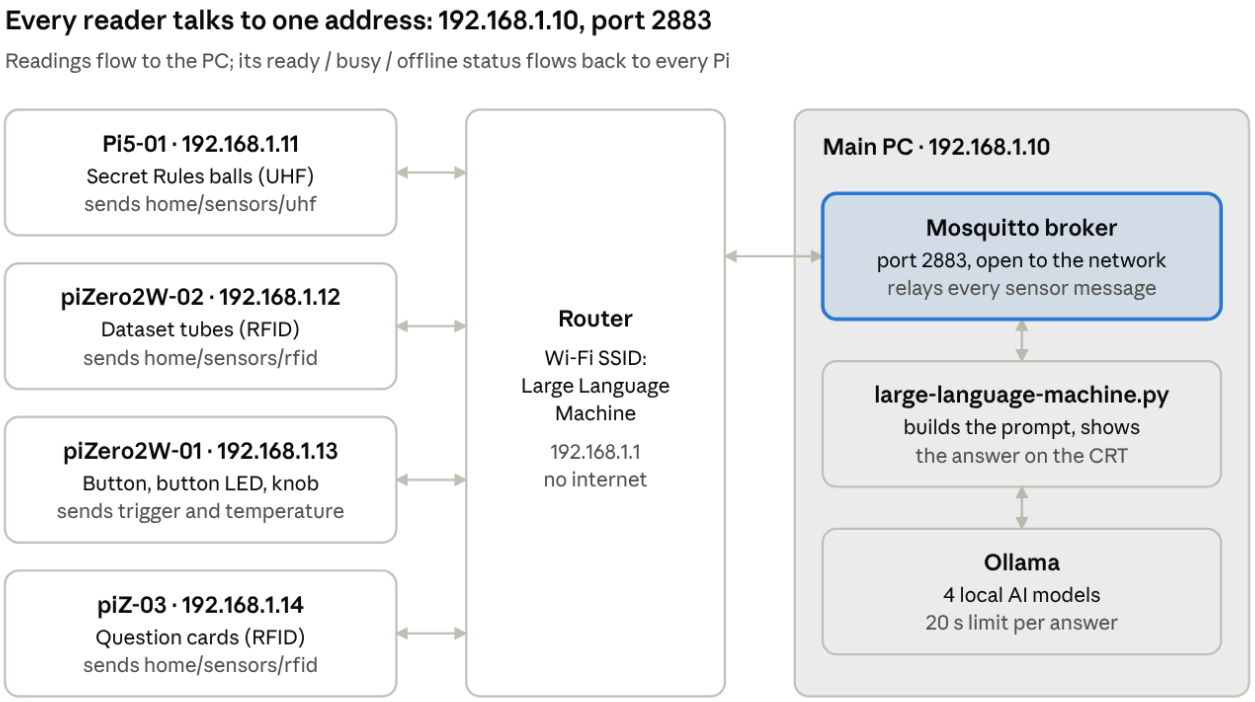
4. **Watch the messages on the PC:**

```
mosquitto_sub -h localhost -p 2883 -t 'home/#' -v
```

   - `Connection Refused: not authorised` means the broker rejects everyone, the script included. Apply the `allow_anonymous true` fix above.
   - `home/sensors/uhf` lines arriving while the screen stays stuck means the script itself has frozen. Press Ctrl+C in its window and re-run it in debug mode (Step 1).
   - `home/system/status offline` and nothing else means the script never sent `ready`. Restart it as in Step 1 and watch its output.

How the machine fits together

The machine is five computers on a private Wi-Fi network, and every part depends on four of them reaching the main PC at **192.168.1.10, port 2883**. If that one address or port is wrong, nothing works, and the screen shows almost nothing because the main script runs with logging switched off.



machine network · 4 readers, router, main PC

If the broker cannot be reached, every startup row shows ❌ at once. A single ❌ points at that one Pi.

- **Router** (inside the machine, SSID `Large Language Machine`) hands out the 192.168.1.x addresses. It runs offline, with no internet.
- **Main PC ("Inference")** runs three things: an MQTT message broker (almost certainly Mosquitto) on port 2883, Ollama (the local AI models), and `large-language-machine.py`, shown full-screen on the CRT.
- **Four Raspberry Pis** read the controls and publish what they see to the broker. They wait for a `ready` message from the main script before doing anything.

Device	Name	IP	Script	Service	Reads
Main PC	Inference	192.168.1.10	`large-language-machine.py`	`launch-ll-machine.service` (user service)	Runs broker, Ollama, display
Pi 5	Pi5-01	192.168.1.11	`uhf-read.py`	`read-uhf.service`	Secret Rules balls (UHF, `/dev/ttyAMA0`)
Pi Zero 2 W	piZero2W-02	192.168.1.12	`rfid-read.py`	`read-rfid.service`	Dataset tubes (RFID, `/dev/ttyS0`)
Pi Zero 2 W	piZero2W-01	192.168.1.13	`button-read.py`, `pot-read.py`	`read-button.service`, `read-pot.service`	Activate button, button LED, Randomness knob
Pi Zero	piZ-03	192.168.1.14	`rfid-read.py`	`read-rfid.service`	Question cards (RFID, `/dev/ttyS0`)

The IPs and the port are written directly into every script. The Pi scripts live in `/home/nathan/large-language-machine/`, so the Pi login is probably `nathan`. Each Pi service waits 150 seconds after boot before starting, which is why the guide says to wait 2 minutes.

The main script needs four Ollama models: `deepseek-r1:7b`, `ministral-3:3b`, `nemotron-3-nano:4b` and `olmo-3:7b-instruct`.

Before you start

Have these to hand:

- [ ] A laptop with Wi-Fi, ideally also an Ethernet cable
- [ ] The keyboard and mouse stored inside the machine
- [ ] A 7 mm allen key to open the panels (how to open each panel is below)

Then do these visual checks and **note what you see**. Each one narrows the fault before you plug anything in.

1. **Do a full reboot**, which takes about 3 minutes:
   a. Press the power button. It is hidden at the rear left of the machine, under the lower panel.
   b. When the screen goes off, wait 5 seconds.
   c. Switch off both wall sockets, marked *1. Compute* and *2. Readers*.
   d. Wait 10 to 15 seconds, so the readers power down completely.
   e. Switch both sockets back on.
   f. Press the power button again, then wait at least 3 minutes before judging anything.

2. **Watch the Activate button light.** It is the best built-in diagnostic, because its pattern is set by the network state:
   - **Repeating pattern (2 slow flashes, 6 fast)**: the button Pi is running but has never heard "ready" from the main PC. Either it cannot reach 192.168.1.10:2883, or the main script is not running.
   - **Solid on**: the button Pi and the main PC are talking. The fault is elsewhere.
   - **Off, never flashed**: the button Pi has no power, has not booted, or its service is not running.
   - **Was on, now off**: the main script stopped (its "offline" message reached the Pi) or the Wi-Fi dropped.
3. **Read the screen.** If it shows the *Startup checks* table, note the title (*Network active* ✅ or *Network inactive* ❌) and which sensor rows are ❌. If it shows a Python error, photograph it. If it shows a blank blue screen or a bare `$` prompt, the main script has exited.
4. **Look at the router.** Its lights should show power and Wi-Fi. If it has no lights, check the *2. Readers* and *1. Compute* sockets.

Opening the panels

Every panel is held by brackets that loosen with the 7 mm allen key.

Panel	How to open it	What it gives access to
Front (small)	Loosen the two brackets on the left and right. Lift the panel up over its lower brackets, pull it towards you and lift it out.	Knob and button wiring, Secret Rules reader (Pi5-01)
Rear	Loosen the two brackets at the top. The panel swings down on its lower hinges.	Main PC, CRT monitor, Dataset tube reader
Left side	Loosen the single bracket at the front (smaller) end. If it will not swing open like a door, also loosen the two hinges at the rear.	Router and main PC
Right side	Same as the left side.	CRT monitor, button and LED wiring

After transport, tighten every scaffold joint again, and check the CRT has not shifted behind the rear panel.

Step 1: Get a shell on the main PC

The screen shows little because the script runs without its debug flag, and its output is not saved to any log file. You get the real messages by stopping it and running it yourself. Use whichever route works.

Route A: keyboard on the PC (no network needed)

1. Open the rear or left side panel (see Opening the panels, above) and plug the internal keyboard and mouse into the PC. The desktop is the same blue as the terminal. To find the menus, press the Super (Windows) key or move the mouse to the dock on the right.
2. Press **F11** to leave full screen. If a Python error is on screen, photograph it now.
3. Press **Ctrl+C** to stop the script. You are left at a `$` prompt in the same terminal.
4. If the screen is frozen or blank, press **Ctrl+Alt+F3** for a text login, sign in, and continue from there. **Ctrl+Alt+F2** (or F1) returns to the desktop.

Route B: SSH from your laptop

1. Join the `Large Language Machine` Wi-Fi, or plug an Ethernet cable into a spare LAN port on the router.
2. Check your laptop got a `192.168.1.x` address. On a Mac: `ipconfig getifaddr en0`.
3. Run `ssh <user>@192.168.1.10`.

Route C: a Pi with no network

Plug a mini-HDMI screen and USB keyboard into it and log in. The services take over `tty1`, so if the console is blank, press **Ctrl+Alt+F2**.

Once you have a shell on the PC, run the script by hand in debug mode:

```
# stop any copy already running (the service only opens the terminal window)
pkill -f large-language-machine.py
cd ~/large-language-machine
source venv/bin/activate
python3 large-language-machine.py debug
```

Debug mode prints every MQTT message received, each prompt sent to Ollama, and each error. Leave it running while you work through Steps 2 to 4 from SSH or a second terminal. To capture it to a file, add `2>&1 | tee ~/llm-debug.log` to the last command.

Step 2: Check the network

The aim is to prove that every device has its exact expected IP and can reach **192.168.1.10:2883**. Work through the checks in this order and stop at the first one that fails: that is your fault.

2a. From your laptop (on the machine's Wi-Fi)

1. **Router client list.** Open `http://192.168.1.1` and find the list of connected or DHCP clients. All five devices should appear with the IPs in the table above.
   - A device missing means it is not on the Wi-Fi (Step 2c).
   - A device on a *different* IP is the likeliest cause of a total failure, because every script points at fixed addresses. A router reset or replacement wipes its address reservations. The fix is to reserve each device's IP again in the router's DHCP settings, then reboot everything.
2. **Ping each device:**

   ```
   for i in 1 10 11 12 13 14; do ping -c 2 -t 2 192.168.1.$i; done
   ```

3. **Check the broker port is open to the network:**

   ```
   nc -vz 192.168.1.10 2883
   ```

   "succeeded" or "open" is good. "Connection refused" means the broker is not running, or only listens on the PC itself (Step 2b).
4. **Watch the traffic.** Install Mosquitto's tools (`brew install mosquitto` on a Mac), then:

   ```
   mosquitto_sub -h 192.168.1.10 -p 2883 -t 'home/#' -v
   ```

Topic	Expect to see	When
`home/system/status`	`ready`, `busy` or `offline`	Straight away (it is retained)
`home/sensors/uhf`	A list of tag IDs, or `"uhf_empty"`	About every 1.5 s
`home/sensors/temperature`	e.g. `[0.4]`	When the knob turns
`home/sensors/rfid`	e.g. `["0E0071A97DAB"]`	When a card or tube goes in
`home/sensors/trigger`	`["TRIGGER"]`	When the button is pressed

A status of `offline` means the main script has stopped, and every reader pauses until it sees `ready`. A topic that never appears points to that one Pi.

5. **Fake a button press** to test the PC on its own:

```
mosquitto_pub -h 192.168.1.10 -p 2883 -t home/sensors/trigger -m
'["TRIGGER"]'
```

2b. On the main PC

```
ip -4 addr                      # must include 192.168.1.10
nmcli device status             # Wi-Fi/Ethernet connected?
systemctl status mosquitto      # broker running?
sudo ss -ltnp | grep 2883       # listening address for 2883
journalctl -u mosquitto -n 50   # broker errors
sudo ufw status                 # a firewall blocking 2883?
```

- The *Network active* title on the startup screen only appears when the PC's own IP is exactly 192.168.1.10.
- `ss` must show `0.0.0.0:2883` (or `*:2883`). If it shows only `127.0.0.1:2883`, the main script can connect but the Pis cannot. Mosquitto 2.x behaves this way unless its config contains `listener 2883` and `allow_anonymous true`. The config is in `/etc/mosquitto/mosquitto.conf` or `/etc/mosquitto/conf.d/`; it is not in the repo.
- Nothing listening on 2883 at all means Mosquitto is stopped, or is on its default port 1883. The main script then fails at once with `ConnectionRefusedError`.

2c. On each Pi (SSH or local screen)

```
hostname; ip -4 addr show wlan0          # right name, right IP?
iwgetid                                  # joined "Large Language Machine"?
ping -c 3 192.168.1.10
python3 -c "import socket; socket.create_connection(('192.168.1.10', 2883),
3); print('broker reachable')"
```

The Pi Zeros only use 2.4 GHz Wi-Fi. If the router's 2.4 GHz band is off, or its SSID or password has changed, they cannot join.

Step 3: Check the services and their logs

Once the network checks pass, check that each program is actually running. The Pi services log to the system journal; the main script does not, which is why Step 1 runs it by hand.

3a. Main PC: the launcher

```
systemctl --user status launch-ll-machine.service
journalctl --user -u launch-ll-machine.service -b
ls ~/large-language-machine/venv/bin/activate    # venv present?
```

- It is a **user** service. It only starts after a desktop session begins, so the PC must log in automatically. If the PC stops at a login screen, the script never starts.
- If the status stays at `activating (start-pre)`, the service is stuck waiting for the display (`xset q` failing).
- If the venv or `~/large-language-machine` is missing or was moved, the terminal opens and closes straight away, leaving a blank desktop.

3b. Main PC: Ollama

```
systemctl status ollama
ollama list                              # all four models listed?
curl http://localhost:11434/api/tags     # API answering?
journalctl -u ollama -n 100              # load errors, out of memory
time ollama run ministral-3:3b "Say hi"  # how long a real answer takes
sudo -n systemctl restart ollama && echo "sudo OK"
```

- Any missing model: `ollama pull <name>`. This needs internet, so connect the PC to another network temporarily.
- The script gives Ollama **20 seconds** per answer, then shows *ERROR! Something went wrong*. If `time` shows a first answer taking longer than that, the machine will error on the first press after each reset.
- The script runs `sudo systemctl restart ollama` on every 60-second reset. If `sudo -n` prints *a password is required*, that restart waits for a password nobody types, and the machine hangs on a blank screen. The fix is a sudoers rule allowing that one command without a password.

3c. Each Pi: its reader service

```
# swap read-rfid for read-uhf, read-button or read-pot
sudo systemctl status read-rfid.service
journalctl -u read-rfid.service -b --no-pager   # since boot
journalctl -u read-rfid.service -f              # live
```

Log line	Meaning
`Broker session established.`	Connected to the PC's broker. Good.

Log line	Meaning
`Server is ready, resuming.` / `Sent: [...]`	Working normally.
`Waiting for main server to be ready...` (and nothing after)	Connected, but no `ready` from the main script. Go to Step 1.
`Broker connection FAILED (rc=...)`	Broker reached but refused the login. Check Mosquitto's `allow_anonymous`.
`DISCONNECTED (rc=...)` repeating	Wi-Fi dropping, or the broker restarting.
`Network not ready yet`	No network on this Pi. Back to Step 2c.
Python traceback naming `/dev/ttyS0` or `/dev/ttyAMA0`	The reader's serial port is unavailable (Step 4).

To restart a reader: `sudo systemctl restart read-rfid.service`. It waits 150 seconds before starting, so be patient. To watch it directly instead, run `sudo systemctl stop read-rfid.service`, then `python3 ~/large-language-machine/rfid-read.py`.

Step 4: Test one sensor at a time

Use this step when the network is fine but one row of the startup table stays ❌. For the reader and knob tests, stop that Pi's service first, so the test script can use the hardware: `sudo systemctl stop <service>`. Afterwards, `sudo systemctl start <service>` puts it back.

Sensor	Pi	Hardware link	Quick test	If it fails
Question cards	piZ-03 (.14)	Serial `/dev/ttyS0`, 9600 baud	`python3 ~/large-language-machine/Tests/RFID/rfid-read.py`, then insert a card. It prints a 12-character ID.	Serial port off, or the console is using it: `sudo raspi-config` → Interface Options → Serial Port → login shell **No**, hardware **Yes**, then reboot
Dataset tubes	piZero2W-02 (.12)	Serial `/dev/ttyS0`, 9600 baud	Same as above, with a tube	Same as above. Also check the reader's wiring behind the rear panel
Secret Rules balls	Pi5-01 (.11)	Serial `/dev/ttyAMA0`, 115200 baud; GPIO pin 7 enables the module	`cd ~/large-language-machine/Tests/RFID && python3 uhf-read.py`	On a Pi 5, `ls -l /dev/ttyAMA0` must exist. Check the UART is enabled in `/boot/firmware/config.txt`
Randomness knob	piZero2W-01 (.13)	MCP3008 chip over SPI, channel 0	`python3 ~/large-language-machine/Tests/knob-test.py`, then turn the knob. The value should move smoothly from 0 to 1.	`ls /dev/spidev0.*` must list devices, else enable SPI in `raspi-config`. If the value jumps, or the knob makes noise while nobody touches it, open the front panel and gently reposition its wires.

Sensor	Pi	Hardware link	Quick test	If it fails
Activate button and LED	piZero2W-01 (.13)	GPIO 24 (button), GPIO 23 (LED)	`journalctl -u read-button.service -f`, then press. Each press logs `button pressed!` or `Button press ignored, server isn't ready.`	Nothing logged at all is wiring. *Ignored* means the main script has not sent `ready` (Step 1)

With the main script in debug mode (Step 1), each read also shows on the PC. A line `*Warning*: unregistered tag: <ID>` means that card, tube or ball is missing from `Data/database.json`. Send that ID to the developer.

Symptom quick reference

Find the row that matches what you see, then jump to the step it names.

What you see	Most likely cause	Go to
Button LED repeats 2 slow + 6 fast flashes; every startup row is ❌	The Pis cannot reach 192.168.1.10:2883: wrong IPs, broker only listening locally, or main script not running	2a, then 2b
Startup screen title says *Network inactive* ❌	The PC is not on 192.168.1.10 (lost reservation, router reset, Wi-Fi not joined)	2a step 1, then 2b
Python error ending in `ConnectionRefusedError`, or a bare `$` prompt	Mosquitto not running, or not on port 2883	2b
Blank blue desktop, no terminal	Launcher did not start: no auto-login, missing venv, or stuck waiting for the display	3a
Some rows ✅, one or two ❌	That Pi: off Wi-Fi, service stopped, or sensor hardware	2c, 3c, 4
Button LED never lights or flashes	Button Pi has no power or has not booted. Check the *2. Readers* socket	2c, 3c
Checks pass, but every press gives *ERROR! Something went wrong*	Ollama takes longer than 20 s to answer (model cold or too slow)	3b
Press gives `Ollama API request failed. Error: ...`	Ollama not running, or that tube's model not installed	3b

What you see	Most likely cause	Go to
Stuck on a blank screen well past 60 s	The automatic Ollama restart is waiting for a `sudo` password	3b
One card, tube or ball does nothing; others work	Tag missing from `Data/database.json`	4

If you are still stuck: what to send the developer

Collect these in one go. With them, the developer can usually find the fault without visiting.

- [ ] The button LED pattern and a photo of the screen after a 3-minute boot
- [ ] A screenshot of the router's client list (names and IPs)
- [ ] The `mosquitto_sub` output from Step 2a while you press the button, turn the knob, and insert a card, a tube and a ball
- [ ] From the PC: `ip -4 addr`, `sudo ss -ltnp | grep 2883`, `ollama list`, and the contents of `/etc/mosquitto/mosquitto.conf` plus `/etc/mosquitto/conf.d/`
- [ ] From the PC: `~/llm-debug.log` from Step 1 (a few minutes of running in debug mode)
- [ ] From each Pi: `journalctl -u <service> -b --no-pager > ~/<service>.log`

To copy a file from a device to your laptop: `scp <user>@192.168.1.10:~/llm-debug.log .` (change the IP and path for each Pi).